

Pilot Deployment Architecture

Hosting the CMR Specialist in the office or on the internet (VM) -
a complete, beginner-friendly guide: what to get, how to set it up,
how to run it daily, and how to fix it

You do not need to be a server expert. This guide assumes you can rent a virtual machine (a computer that lives on the internet instead of your desk), copy files to it, and run commands by pasting them. Every step tells you what you are doing and why. By the end, the system runs day and night without your laptop, staff use it from their browsers, data is backed up, and you know exactly what healthy looks like.

1. The Big Picture (What You Are Building)

Today the system runs on a laptop: it sleeps when the lid closes and dies with the disk. The pilot moves it onto one always-on virtual machine (VM) - a rented computer in a data center, or a small server in the office. Five programs run on it:

Program	Job	Who can reach it
Caddy (front door)	Encrypts traffic (HTTPS), forwards browsers to the app	Everyone (the only public face)
Web (the app)	Console, admin site, API	Only Caddy (hidden inside)
Worker (the engine)	Runs queued tasks in the background	Nobody directly - it reads the database
Postgres (the safe)	Stores everything durably	Only web + worker
Backup (the photocopier)	Copies the database nightly, prunes old copies	Writes to its own disk + offsite

Think of it as an office: Caddy is the receptionist at the single entrance, web serves the counter, worker toils in the back room, Postgres is the safe, backup photocopies the ledger every night. Staff never walk past reception.

2. What You Need Before You Start

2.1 The virtual machine

- **Size:** 2 virtual CPUs, 4-8 GB memory, 40 GB disk. This system idles near zero; the headroom is for the database and uploaded PDFs. Anything sold as a 'small' server qualifies.
- **System:** Ubuntu 22.04 LTS (long-term support - boring and stable, which is what you want).
- **Where:** any provider (AWS, Azure, DigitalOcean, Hetzner, or a local host) - or a mini-PC/old desktop in the office running Ubuntu. The architecture is identical; only the address changes.
- **Cost:** a small VM rents for roughly \$6-12/month; office hardware is free if spare. AI usage stays at cents per day.

2.2 A name (for internet hosting)

If staff reach it over the internet, you want *cmr.youroffice.com* instead of bare numbers. Buy or use an existing domain, then create one DNS 'A record' pointing that name at the VM's public IP. This takes 10 minutes at any registrar and up to an hour to spread worldwide. Pure office-LAN pilots can skip this and use the private IP (e.g. 192.168.1.50) - but then there is no encrypted certificate from the internet, so prefer a real name even internally if you can.

2.3 Secrets you must prepare (never in git)

- Your OpenRouter API key (the INFERENCE_API_KEY you already have).
- A long random database password (generate: *openssl rand -hex 24*).
- A first admin username + strong password (you will create it on first boot).
- Keep these in a password manager, and later in the VM's .env file only.

3. Setting Up the VM (Step by Step)

1. **Log in.** `ssh user@YOUR-VM-IP`. You are now typing on the rented computer. Everything below happens there.
2. **Install the two tools.** Docker (runs the five programs as sealed boxes) and Git (fetches the code). Ubuntu commands: `sudo apt update && sudo apt install -y docker.io docker-compose-plugin git`, then log out and back in so Docker permissions apply.
3. **Fetch the deployable line.** `git clone https://github.com/jimthejanga/cmr-specialist.git && cd cmr-specialist && git checkout next/stage2`. Note: the VM runs the `next/stage2` line, never `main` - `main` stays the untouched proven engine.
4. **Write the secrets file.** Copy `.env.example` to `.env` and fill the real values: database password, inference key, domain name. Set permissions: `chmod 600 .env` (only you can read it).
5. **Open the firewall.** Allow ports 80 and 443 (web) and 22 (your SSH only): `sudo ufw allow 80,443/tcp && sudo ufw allow from YOUR-OFFICE-IP to any port 22 && sudo ufw enable`. Everything else stays closed - the database port is never exposed.
6. **Start everything.** `docker compose -f docker/docker-compose.yml up -d --build`. Five containers rise: db, web, worker, backup, caddy. Check: `docker compose ps` (all 'running') and `curl -s http://localhost:8080/health` (status ok).
7. **Create the first admin safely.** Over your SSH connection (encrypted tunnel, never the open web), create the first user - it auto-becomes admin - then sign in at `https://YOUR-DOMAIN/admin` and confirm the ADMIN badge. Create each staff member's login while signed in as admin.
8. **Prove the backup.** Run the backup once by hand, list the backup volume (one fresh timestamped file), then practice a restore on a scratch copy. A backup you have never restored is a rumor, not a backup.

Total: an afternoon the first time, under an hour once practiced.

4. How the Pieces Fit (Architecture Detail)

4.1 Traffic path

```
staff browser --HTTPS(443)--> Caddy --HTTP--> web:8080 --SQL--> Postgres
worker --SQL--> Postgres backup --SQL--> Postgres (dump only)
NOBODY --> Postgres directly (no published port except inside compose)
```

Caddy gets its certificate automatically (Let's Encrypt) the first time your domain points at it - no manual certificate work. HTTP on port 80 exists only to redirect to HTTPS.

4.2 Where state lives

State	Lives in	Survives restart?	Backed up?
Cases, users, tasks, audit	Postgres volume <code>pg_data</code>	yes	yes (nightly dump)
Nightly dumps (14 days)	Volume <code>pg_backups</code> + offsite copy	yes	that IS the backup
Uploaded knowledge PDFs	<code>harness_data</code> volume	yes	include in offsite copy
Admin test inserts (overlay)	<code>var-equivalent</code> volume	yes	no (test data, reseedable)
Secrets (<code>.env</code>)	VM file, root-only	yes	NO - kept in password manager

4.3 Releases and rollback (no-merge rule)

The VM runs `next/stage2` image tags (`vm-shed-1`, `vm-shed-2...`). A deploy is: pull, rebuild, restart - two commands, under two minutes of downtime (or zero with a second web replica). A rollback is: restart the previous tag - one command. Fixes from the proven `main` line arrive only by deliberate cherry-pick, never by merge, so the office never inherits an accidental change. Each deploy is recorded (tag + date + what changed) in a one-line release log.

5. Daily Operations (The Routine)

5.1 Morning (30 seconds, anyone)

- Open the admin Worker page: heartbeat fresh, queue near zero, no (fallback) badge clusters.
- If the queue age climbs: the worker is down - restart that one container, watch it drain.

5.2 Staff routine

- Sign in with personal logins; never share. Log out on shared machines.
- Payments first (read-only, safest), ownership second; WRITE-gated work only after the approval habit exists.
- Fallback badge means 'the AI was quiet' - double-check that result before acting.

5.3 Weekly (admin, 10 minutes)

- Confirm a fresh backup file exists AND the offsite copy timestamp is current.
- Glance at disk use (uploads accumulate) and the approval log for anomalies.
- Rotate nothing unless someone left - then disable their account immediately.

6. When Things Break (Troubleshooting for Beginners)

Symptom	Likely cause	Fix
Site won't load	VM off / Caddy down / DNS wrong	Provider console: VM running? Then: compose ps; dig YOUR-DOMAIN (IP match?)
Login fails for everyone	DB container down	compose ps; restart db, then web+worker (order matters)
Tasks stuck queued	Worker down	Admin Worker page heartbeat stale - restart worker container only
Certificate warning	Brand-new domain / DNS just changed	Wait up to an hour; Caddy retries automatically
Disk full	Uploads or old backups	Prune backups past 14 days; move old knowledge PDFs to archive
Forgot admin password	Human memory	SSH in, use the password-reset procedure (documented in runbook); never email passwords

Golden rule: containers are disposable, volumes are precious. Never delete a volume to 'fix' something - restart the container instead. Data loss comes from volume commands, never from restarts.

7. Costs, Limits, and What Comes Next

Item	Pilot scale	Notes
VM + domain	~\$6-12/month + ~\$10/year	office hardware: free
AI inference	cents/day	flash-class model, paced + cached
Staff time	30-sec mornings + 10-min weeklies	plus two shadow weeks at launch
Capacity	5-20 staff, hundreds of cases/day	single web + single worker; second web replica is the first scale step

What this pilot deliberately excludes: citizen portal, supervisor dashboards, live-update plumbing, second API version, microservices. After a month of real use, staff requests - not guesses - decide what gets built next, on the *next/stage2* line, behind a new image tag.